

REACT JS • PRACTICAL ACTIVITY

# React Activity Portal

A single React application, five activities, one shared Navbar.

1

Login

2

Grade Eval.

3

Password

4

Electric Bill

5

Attendance

*State · Events · Conditional Logic · Conditional Rendering · UI Design*

**Grading: Design 30% · Logic & Functionality 70%**

## OVERVIEW

# What You'll Build

You will build a single React application called the **React Activity Portal**. It contains five separate activities, each on its own page, all reachable from one shared Navbar — so the finished project feels like one cohesive app, not five disconnected demos.

### Every activity page must demonstrate:

- useState
- onChange / onClick / onSubmit
- if / else if / else
- Conditional rendering
- Input validation

1

Login Authentication

2

Student Grade Evaluation

3

Password Strength Checker

4

Electricity Bill Calculator

5

Employee Attendance Checker

## GENERAL REQUIREMENTS

# One Project, Five Pages

- All five activities live inside one React project — never five separate projects.
- Each activity gets its own page or component, clearly separated from the others.
- You choose the file organization, as long as the five activities stay clearly separated.

**Suggested — not required:** organize your own way as long as every activity is its own component.

```
src/  
├── components/  
├── pages/  
│   ├── Login.jsx  
│   ├── GradeEvaluation.jsx  
│   ├── PasswordChecker.jsx  
│   ├── ElectricityBill.jsx  
│   └── AttendanceChecker.jsx  
├── App.jsx  
└── main.jsx
```

## GENERAL REQUIREMENTS

# The Navigation Bar

*Visible and usable on every page of the application.*

React Activity Portal

Home

Activity 1

Activity 2

Activity 3

Activity 4

Activity 5

### Must include

Home

Activity 1

Activity 2

Activity 3

Activity 4

Activity 5

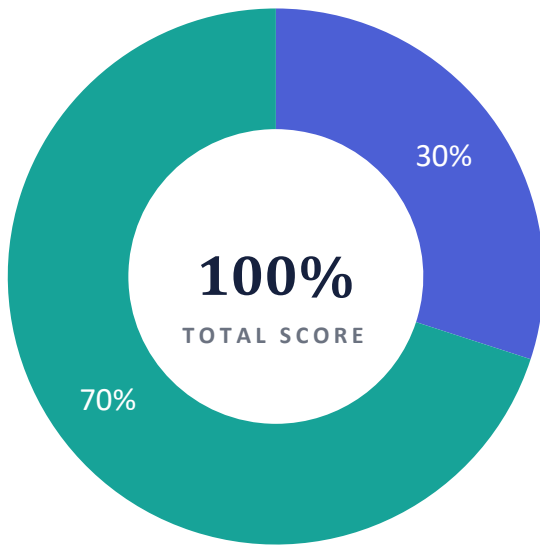
### Navigation rules

- React Router taught in class? You may use it.
- Not taught yet? Use React state to conditionally render the active page instead.
- Write every nav link individually and directly in JSX.
- Do not use `map()`, an array of objects, or any loop to build the Navbar.

## GRADING

# Grading at a Glance

*Functionality outweighs visual design. A beautiful UI with broken logic will not score well.*



■ Design ■ Logic & Functionality

## 30%

### DESIGN

Navbar, layout, color & type, forms & buttons, responsiveness, overall UI/UX.

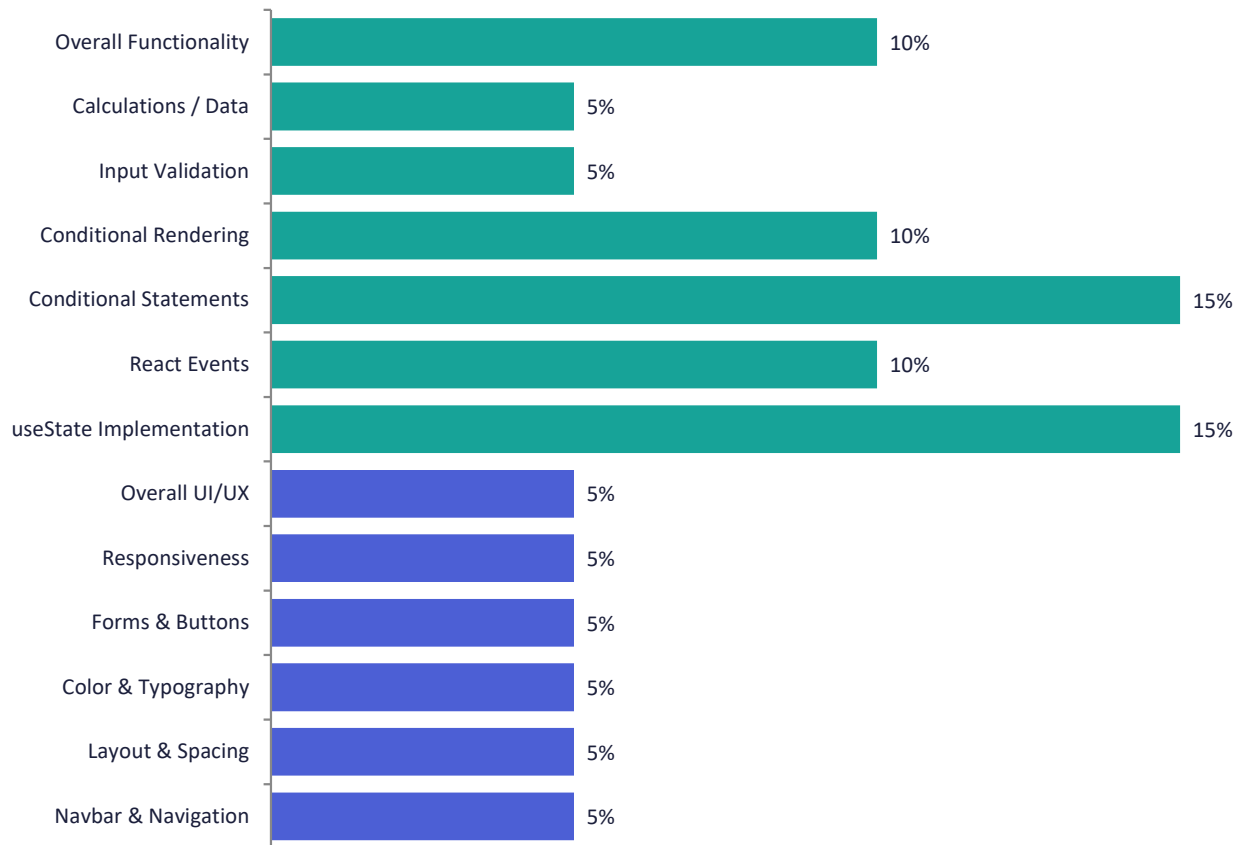
## 70%

### LOGIC & FUNCTIONALITY

useState, React events, conditional statements & rendering, validation, calculations, overall functionality.

## GRADING

# Full Grading Breakdown



**Design — 30%**

**Logic & Functionality — 70%**

Six design criteria at 5% each; seven logic criteria weighted by depth of reasoning required.

**DESIGN = 30%**

**LOGIC = 70%**

**TOTAL = 100%**

*A beautiful interface with broken functionality will not receive a high score.*

# 1

## ACTIVITY 1

# Login Authentication

*Username + password form with validation and a logged-in state.*

### INPUTS & BUTTONS

- Username input
- Password input
- Login button
- Logout button (after success)

### CONDITIONS

**Both fields empty** → "Please enter username and password."

**Credentials correct** → "Login successful!"

**Credentials incorrect** → "Invalid username or password."

**After success** → hide form, show welcome + Logout

**Logout clicked** → return to login form

### RESULT PANEL SHOWS

- › Welcome message
- › Logout button
- › Validation / error text

### DEMONSTRATES

useState

Form events

Validation

if / else

Conditional rendering

## 2

## ACTIVITY 2

# Student Grade Evaluation

Evaluate a score into Excellent → Failed, with range validation.

## INPUTS & BUTTONS

- Student Name input
- Score input
- Evaluate button
- Clear button

## CONDITIONS

**90 – 100** → Excellent

**85 – 89** → Very Good

**80 – 84** → Good

**75 – 79** → Passed

**Below 75** → Failed

**< 0 or > 100** → “Invalid score”

## RESULT PANEL SHOWS

› Student Name

› Score

› Remarks

## DEMONSTRATES

useState

onChange

onClick / onSubmit

Input validation

if / else if / else

Conditional rendering



## 3

## ACTIVITY 3

# Password Strength Checker

Classify a password by length as Weak, Medium, or Strong.

## INPUTS & BUTTONS

- Password input
- Check Password button
- Clear button

## CONDITIONS

**Empty** → "Please enter a password."

**< 6 characters** → Weak Password

**6 – 9 characters** → Medium Password

**≥ 10 characters** → Strong Password

**Strong (≥ 10)** → "Status: Strong – You can use this password."

**Otherwise** → "Status: Weak – Create a stronger password."

## RESULT PANEL SHOWS

- › Password Status
- › Strength message
- › Visual strength indicator

## DEMONSTRATES

useState

onChange

onClick

String length

if / else if / else

Conditional rendering

## 4

## ACTIVITY 4

# Electricity Bill Calculator

Compute a bill from kWh consumption across tiered rates.

## INPUTS & BUTTONS

- Customer Name input
- Consumption (kWh) input
- Calculate Bill button
- Clear button

## CONDITIONS

**0 – 100 kWh** → ₱10 per kWh

**101 – 200 kWh** → ₱12 per kWh

**201 – 300 kWh** → ₱15 per kWh

**Above 300 kWh** → ₱18 per kWh

**Bill ≥ ₱5,000** → High Electricity Usage

**Bill < ₱5,000** → Normal Electricity Usage

## RESULT PANEL SHOWS

- › Customer Name
- › Consumption
- › Rate Applied
- › Total Bill
- › Usage Status

## DEMONSTRATES

useState

onChange

onClick

Numeric input

Calculations

Multiple conditions

Conditional rendering

## 5

## ACTIVITY 5

# Employee Attendance Checker

*Classify a decimal time-in value as On Time, Late, or Very Late.*

## INPUTS & BUTTONS

- Employee Name input
- Time In input (e.g. 8.5 = 8:30 AM)
- Check Attendance button
- Reset button

## CONDITIONS

**Time In  $\leq 7$**  → On Time

**$8 < \text{Time In} \leq 8$**  → Late

**Time In  $> 8$**  → Very Late

**On Time** → "Status: On Time – Good job!"

**Late** → "Status: Late – Please be on time tomorrow."

**Very Late** → "Status: Very Late – Report to your supervisor."

## RESULT PANEL SHOWS

- › Employee Name
- › Time In
- › Attendance Status
- › Follow-up message

## DEMONSTRATES

Multiple useState vars

onChange

onClick

Numeric validation

if / else if / else

Conditional rendering

## FINAL STRUCTURE

# How the Portal Should Behave

*A persistent Navbar up top; the content area swaps between Home and each activity.*

```
-----  
| React Activity Portal |  
| Home | Activity 1 | Activity 2 | Act. 3 |  
|      | Activity 4 | Activity 5 |      |  
-----  
  
Activity Page  
  
[ Activity Content ]  
[ Form / Inputs ]  
[ Buttons ]  
[ Result / Feedback ]
```

Navbar stays visible on every page — Home + all 5 activities.

Content area below it swaps per selection (state or router).

Each activity keeps its own inputs, buttons, and result panel.

User never leaves the single app to move between activities.